

ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук,
должность, уч. степень, звание

подпись, дата

Н.А. Соловьева
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №2

Объектно-ориентированное проектирование программного обеспечения.
Разработка диаграммы последовательности

по курсу: Теория автоматов

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4341В

подпись, дата

Р.С. Кулаков
инициалы, фамилия

Санкт-Петербург 2025

1 Вариант задания

В качестве разрабатываемой системы выбрана предметная область с прошлого семестра – Облачное хранилище файлов

2 Описание сценария диаграммы последовательности «Скачать файл»

2.1 Участники (линии жизни):

- Пользователь — инициатор запроса на скачивание
- FileController — контроллер/сервис, который координирует процесс скачивания
- StorageService — сервис хранилища, проверяет наличие и права.
- Journal — система журналирования, логирует ключевые события.

2.2 Пошаговое описание сценария

2.2.1 Инициирование

Пользователь посылает сообщение Скачать файл контроллеру. Контроллер сразу записывает в журнал событие «запрос_скачивания».

2.2.2 Проверка существования файла

FileController обращается к Storage с запросом «Проверить существование файла». Хранилище возвращает результат проверки.

2.2.3 Ветвление — файл найден / не найден (фрагмент alt с break)

Если файл не найден, выполнится ветвь ошибки: контроллер сообщает пользователю об ошибке «файл не найден» и логирует это событие. В диаграмме используется break, что семантически означает досрочное завершение дальнейшего сценария — дальнейшие шаги не выполняются.

Если файл найден, сценарий идёт дальше — проверяется доступ (следующий шаг).

2.2.4 Проверка прав доступа

FC запрашивает у Storage информацию о правах доступа пользователя на скачивание, Storage возвращает результат проверки.

2.2.5 Альтернатива — доступ разрешён / запрещён (фрагмент alt)

Если доступ запрещён — FC возвращает пользователю ошибку «загрузка запрещена» и логирует это событие. Сценарий завершается в этой ветви.

Если доступ разрешён — переходим к финальным действиям.

2.2.6 Опциональное действие после скачивания (фрагмент opt)

В ветви, где доступ разрешён, присутствует опциональная секция: если пользователь выбрал опцию «Открыть после скачивания», контроллер инициирует локальное открытие файла. Это действие выполняется только при выполнении условия (выбор пользователя).

2.2.1 Завершение

После успешного завершения контроллер логирует событие скачивание_успешно и уведомляет пользователя о завершении скачивания.

2.3 Используемые комбинированные фрагменты

2.3.1 break — прерывание сценария: в вашей диаграмме применяется при ситуации «файл не найден», чтобы показать, что дальнейшие шаги не выполняются.

2.3.2 alt — альтернативные ветви: используется для разделения логики по результатам проверок (существует/не существует, доступ разрешён/запрещён).

2.3.3 opt — опциональная ветвь: показывает необязательное действие «Открыть после скачивания», которое выполняется только если пользователь выбрал эту опцию.

3 Описание сценария диаграммы последовательности «Загрузить файл»

3.1 Участники (линии жизни)

- Пользователь — инициатор запроса на скачивание
- FileController — контроллер/сервис, который координирует процесс скачивания
- StorageService — сервис хранилища, проверяет наличие и права.
- Journal — система журналирования, логирует ключевые события.
- VirusScanner — антивирусный сервис, проверяет загруженный файл на угрозы.
- Converter — компонент обработки файла (конвертация/индексация/генерация миниатюр).

3.2 Пошаговое описание сценария

3.2.1 Инициирование загрузки

Пользователь отправляет Инициировать загрузку в FileController. FileController записывает событие загрузка в Journal. FileController запрашивает у StorageService квоту для пользователя (Запрос квоты). StorageService возвращает информацию о квоте.

3.2.2 Проверка квоты (ветвление / досрочное завершение)

Если квоты недостаточно, выполняется ветвь ошибки: FileController возвращает пользователю ошибку Ошибка: квота превышена. FileController записывает в Journal событие недостаточно_квоты. Процесс загрузки прерывается (логический break). Если квоты достаточно, процесс продолжается к передаче частей.

3.2.3 Передача частей файла (цикл — for each)

Для каждой части файла (for each chunk in file.chunks): Пользователь отправляет часть контроллеру. FileController пересылает часть в StorageService для сохранения. StorageService возвращает результат сохранения. Если при сохранении части возникает ошибка, срабатывает прерывание: FileController уведомляет пользователя

Ошибка: ошибка при загрузке файла. FileController записывает в Journal событие ошибка_загрузки. Цикл и весь сценарий завершатся досрочно (break).

3.2.4 Сборка частей в файл

После успешного сохранения всех частей StorageService собирает части в единый файл и возвращает Загруженный файл в FileController.

3.2.5 Параллельная обработка и антивирус (фрагмент par)

После сборки запускаются параллельные ветви:

Ветка 1 — обработка файла (Converter):

FileController передаёт файл в Converter для обработки (конвертация, индексация и т.п.). По завершении обработки Converter возвращает результат; FileController записывает в Journal событие обработка_успешна.

Ветка 2 — антивирусная проверка:

FileController инициирует проверку файла в VirusScanner.

Если антивирус сообщает, что файл заражён, выполняется прерывание: FileController даёт команду StorageService удалить файл; в Journal записывается событие файл_заражен; пользователю отправляется ошибка Ошибка: попытка загрузить зараженный файл; дальнейшие действия отменяются (break в параллельном блоке).

Если файл чист, FileController посылает в StorageService команду пометить файл как безопасный.

3.2.6 Опциональная публикация ссылки (фрагмент opt)

Если пользователь выбрал создание публичной ссылки: FileController просит StorageService создать публичную ссылку; StorageService возвращает URL; FileController активирует/зарегистрирует ссылку. Если опция не выбрана — этот шаг пропускается.

3.2.7 Завершение

FileController записывает в Journal событие загрузка_успешна. FileController уведомляет пользователя Уведомить: загрузка завершена.

3.3 Используемые комбинированные фрагменты

- 3.3.1 loop (for each) — цикл по частям файла: каждую часть отправляют и сохраняют повторяющимся образом до окончания набора частей. В диаграмме реализовано как loop for each chunk in file.chunks.
- 3.3.2 break — прерывание сценария: в вашей диаграмме применяется при ситуации «файл не найден», чтобы показать, что дальнейшие шаги не выполняются.
- 3.3.3 par (параллельность) — параллельное выполнение двух ветвей: обработка файла (Converter) и антивирусная проверка (AV) выполняются одновременно, независимо друг от друга.
- 3.3.4 opt (опция) — необязательное поведение: создание публичной ссылки выполняется только при выборе пользователем этой опции.

4 Диаграммы последовательности

4.1 Диаграмма последовательности «Скачать файл»

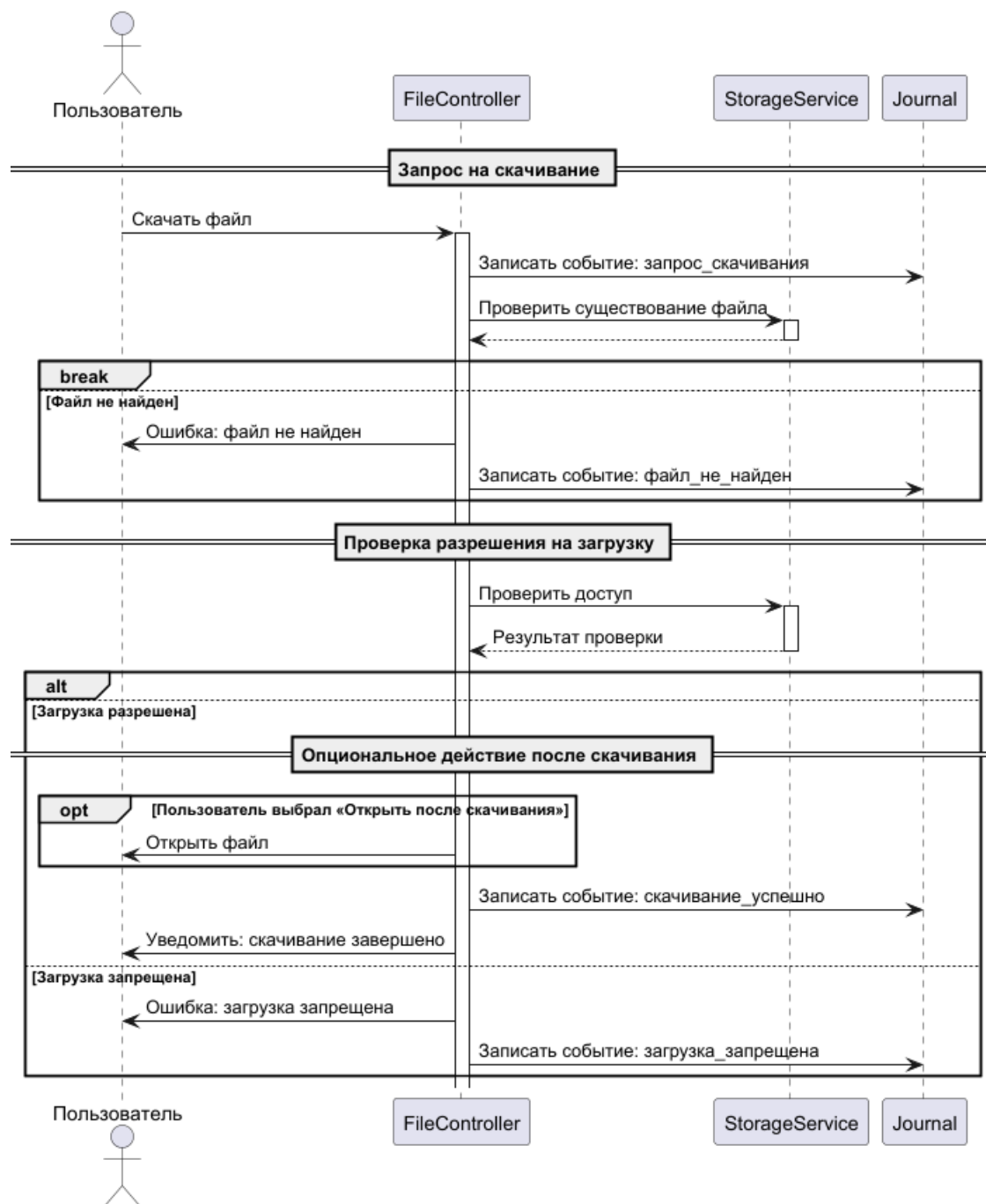


Рисунок 1 – Диаграмма последовательности «Скачать файл»

4.2 Диаграмма последовательности «Загрузить файл»

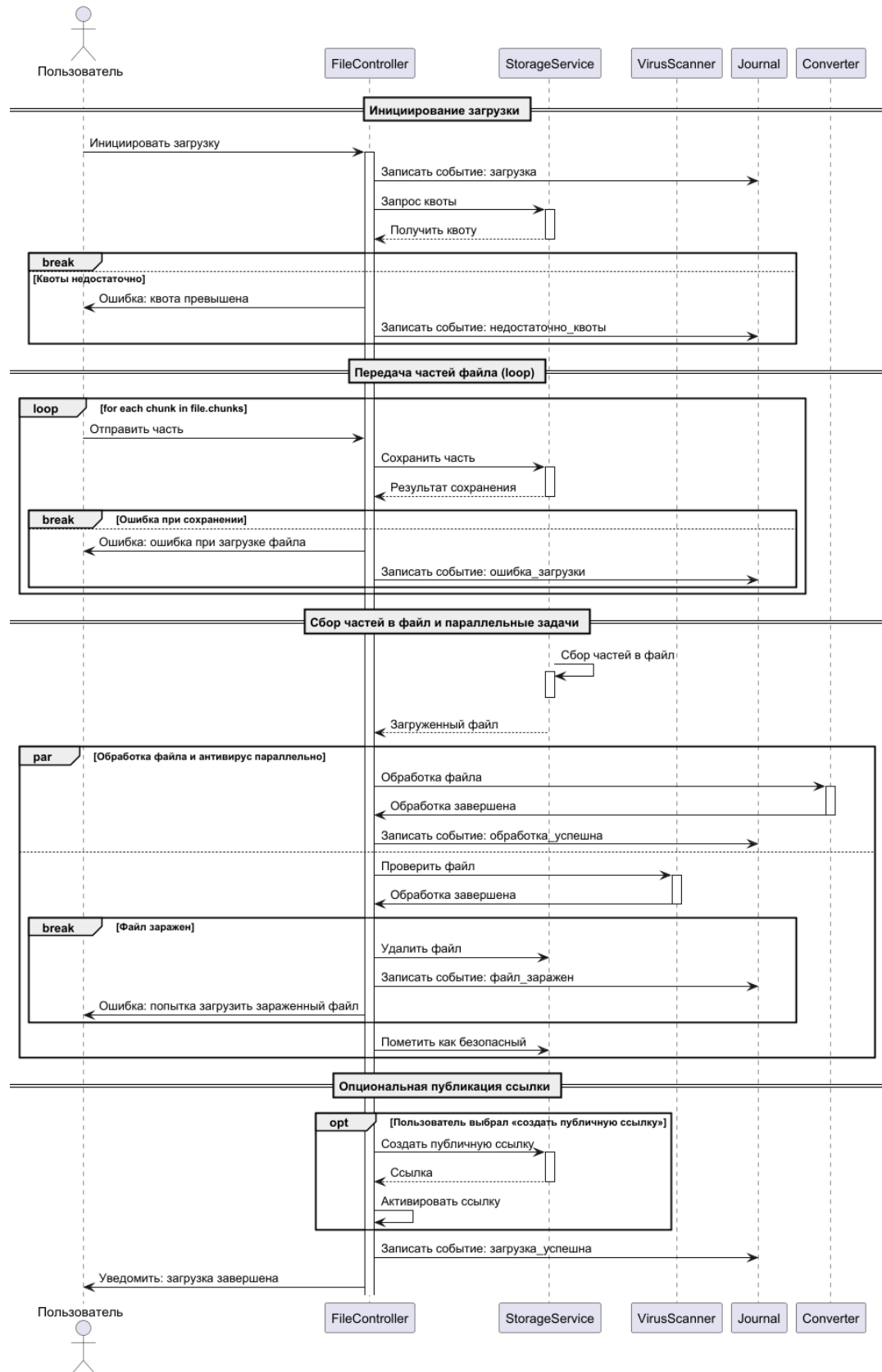


Рисунок 2 - Диаграмма последовательности «Загрузить файл»

5 Анализ выполненной работы

Диаграмма построена корректно: выбранные участники отражают реальные роли в системе (пользователь, контроллер, хранилище, антивирус, обработчик и журнал). Использованные все комбинированные фрагменты:

- Завершение (break)
- Цикл (loop)
- Альтернатива (alt)
- Вариант (opt)
- Параллельность (par)

Они применены уместно и помогают показать разные сценарии выполнения — от успешных путей до ошибок и параллельных процессов.

Алгоритм в целом точный: он включает проверку квоты, передачу файла, сборку файла, обработку, антивирусную проверку и опциональное создание ссылки. Ошибки квоты, сохранения или заражения файла корректно завершают процесс с помощью break.

Есть несколько моментов, которые можно улучшить.

1. В цикле передачи частей (loop) можно добавить уточнение о повторных попытках (retry) или реакции на частичные сбои.
2. Можно более явно показать, что происходит с уже загруженными частями при ошибке.